

Reviewer Pack — Frobenius Fourier Defect of Barrington AND_n PBP Trajectory

Entry 51aafa940751 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 03:06:59 UTC

Frobenius Fourier Defect of Barrington AND_n PBP Trajectory

Entry ID: 51aafa940751 Verdict: INCONCLUSIVE Recorded: 2026-05-19 03:06:59 UTC

1. Conjecture

Field A (mathematical branch): Non-abelian harmonic analysis / Diaconis-Shahshahani Fourier theory on the symmetric group S_5 (Diaconis 1988 ‘Group Representations in Probability and Statistics’; Diaconis-Shahshahani 1981; Saloff-Coste 1997) **Field B** (complexity object): Width-5 Barrington permutation branching programs (Barrington 1989) realizing NC^1 / Ben-Or-Cleve 1992 formula-to-ABP simulation

Statement:

Let P_n be Barrington’s canonical width-5 PBP for AND_n on $n = 2^k$ inputs ($k \geq 2$), built recursively by the depth-1 commutator program $[a(x_1), b(x_2)]$ for AND_2 using fixed non-commuting 5-cycles $a=(1\ 2\ 3\ 4\ 5)$, $b=(1\ 3\ 5\ 2\ 4)$, nested via the standard Barrington substitution scheme to length $L(n)=4^{k=n/2}$. For each input $x \in \{0,1\}^n$ let $\pi_0(x)=e$, $\pi_i(x)=g_1(x) \dots g_i(x) \in S_5$ be the prefix products and let $\bar{M}(P_n, x) := (1/L(n)) \cdot \sum_{i=1}^{L(n)} \text{Perm}(\pi_i(x)) \in \mathbb{R}^{5 \times 5}$ be the empirical average of the 5×5 permutation-representation matrices along the trajectory. Define the Frobenius Fourier defect $\bar{D}(P_n) := \mathbb{E}_x \|\bar{M}(P_n, x) - (1/5) \cdot J\|_F^2$ averaged over 30 uniform-random inputs $x \in \{0,1\}^n$. Then for all $n \in \{4, 8, 16, 32\}$ the sequence $\bar{D}(P_n)$ is strictly monotone non-increasing in n , and every consecutive log-log decay slope $\alpha_n := \log_2(\bar{D}(P_{n/2})/\bar{D}(P_n))$ lies in the open interval $(0.5, 3.0)$. A single n where monotonicity or the slope band fails falsifies the conjecture.

Rationale (proposer’s reasoning):

Barrington’s PBP for AND_n is a rigid nested-commutator scaffold in S_5 , structurally different from an i.i.d. random walk; Diaconis-Shahshahani would predict Frobenius defect $\Theta(1/L) = \Theta(1/n^2)$ for a random walk of length $L=n^2$, so any observed slope strictly below 2 detects the commutator’s structural memory and gives a non-trivial structural fingerprint of formula depth $d = \log_2 n$. The Frobenius L^2 ratio uses real Schatten-2 norm with no F_q polynomial extension (ring lifts of permutation matrices do not preserve $\|\cdot\|_F^2 / \|\cdot\|_F^2$ ratios), so the bridge is Aaronson-Wigderson algebrization-safe; $\bar{D}$ depends on the PBP wiring (two PBPs computing AND_n can have very different $\bar{D}$), shielding from Razborov-Rudich.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 11c0ee3463fad10b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Define permutations as tuples (e.g., (1, 2, 3, 4, 5))
a = (1, 2, 3, 4, 5)
b = (1, 3, 5, 2, 4)

def compose(p1, p2):
    return tuple(p1[p2[i] - 1] for i in range(5))

def inverse(p):
    return tuple(p.index(i + 1) + 1 for i in range(5))

def perm_matrix(p):
    M = [[0] * 5 for _ in range(5)]
    for i, j in enumerate(p):
        M[i][j - 1] = 1
    return M

def frobenius_norm(M):
    return math.sqrt(sum(x**2 for row in M for x in row))

def Barrington_AND(n):
```

```

if n == 2:
    return [(1, a, b), (2, b, a)]
else:
    subprogram = Barrington_AND(n // 2)
    new_program = []
    for literal_index, perm_for_x0, perm_for_x1 in subprogram:
        new_perm_for_x0 = compose(a, perm_for_x0) if literal_index % 2 == 0 else inverse(compose(a, perm_for_x0))
        new_perm_for_x1 = compose(b, perm_for_x1) if literal_index % 2 == 0 else inverse(compose(b, perm_for_x1))
        new_program.append((literal_index * 2 + 1, new_perm_for_x0, new_perm_for_x1))
    return new_program

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 8, 16, 32]
    results = []

    for n in n_values:
        L_n = n ** 2
        total_defect = 0

        for _ in range(30):
            x = tuple(random.randint(0, 1) for _ in range(n))
            prefix_products = [a] * (n + 1)

            for literal_index, perm_for_x0, perm_for_x1 in Barrington_AND(n):
                if x[literal_index - 1] == 0:
                    prefix_products.append(perm_for_x0)
                else:
                    prefix_products.append(perm_for_x1)

            M_bar = sum(perm_matrix(p) for p in prefix_products[1:]) / L_n
            J = [[Fraction(1, 5)] * 5 for _ in range(5)]
            defect = frobenius_norm([M_bar[i][j] - J[i][j] for i in range(5) for j in range(5)])
            total_defect += defect

        avg_defect = total_defect / 30
        results.append(avg_defect)

    D_bar_values = [results[0]]
    slopes = []

    for i in range(1, len(results)):
        slope = math.log2(results[i - 1] / results[i])
        if not (0.5 < slope < 3.0):
            return {
                "metric_name": "D_bar",
                "metric_value": None,
                "instances_tested": 4 * 30,
                "conjecture_holds": False,
                "counterexample": f"slope_{n_values[i]}={slope}"
            }
        D_bar_values.append(results[i])
        slopes.append(slope)

    if all(D_bar_values[i] > D_bar_values[i + 1] for i in range(len(D_bar_values) - 1)):
        return {

```

```

        "metric_name": "D_bar",
        "metric_value": None,
        "instances_tested": 4 * 30,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "D_bar",
        "metric_value": None,
        "instances_tested": 4 * 30,
        "conjecture_holds": False,
        "counterexample": "monotonicity_failed"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    all_results = []
    total_defects = 0
    count_supporting = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        all_results.append(trial_result["metric_value"])
        if trial_result["conjecture_holds"]:
            count_supporting += 1

        total_defects += sum(all_results) / len(all_results)

    mean_defect = total_defects / len(seeds)
    support_fraction = count_supporting / len(seeds)

    if all(r is not None for r in all_results):
        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_defect} std=0 support_fraction={support_fraction}")
        else:
            print(f"RESULT: FALSIFIED counterexample='monotonicity_failed' first_failing_seed={seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5a7bd9ab.py", line 117, in <module>
    trial_result = run_trial(seed)
                  ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5a7bd9ab.py", line 63, in run_trial

```

```
if x[literal_index - 1] == 0:
    ~~~~~
```

IndexError: tuple index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing data, preventing verification of conjecture
| next: Fix index out-of-range error in test code by validating tuple bounds

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	450963
2	preregistration	ollama_remote	qwen3:8b	0	0	30016
3	novelty	ollama_remote	qwen3:8b	0	0	24876
4	novelty	ollama_remote	qwen3:8b	0	0	22514
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16601
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13495
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12494
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13868
9	judge	ollama_remote	qwen3:8b	0	0	21333

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 606161 ms total latency. Provider mix: {'claude_max': 1, 'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/51aafa940751.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/51aafa940751.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/51aafa940751.tar.gz` (if generated)